



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Criptografía y Seguridad Computacional - IIC3253
Rúbrica Tarea 4

Preguntas

1. Responda las siguientes preguntas.

- (a) [0.5 puntos] Para el caso de cuatro participantes, demuestre que si el participante 3 construye una firma $(s_1, s_2, s_3, s_4, c_1, f)$ de un mensaje m utilizando el protocolo, entonces el proceso de verificación concluye que la firma es válida.
- (b) [0.5 puntos] Un participante i podría tratar de ocultar que generó dos firmas utilizando $f = T^{x_i}$ en una de las firmas, y un elemento arbitrario $f' \in G$ con $f' \neq f$ en la otra firma. Considerando nuevamente el caso con cuatros participantes, explique por qué el participante 1 no podría generar una firma válida si utiliza un elemento $f' \in G$ tal que $f' \neq T^{x_1}$.
- (c) [0.5 puntos] Demuestre que $|\langle T \rangle| = q$.
- (d) [0.5 puntos] Suponga que modificamos el protocolo de manera que el elemento T usado en el protocolo es generado por una tercera parte; por ejemplo, este elemento T podría ser generado por el Servel en caso de una elección o por los profesores del curso en el caso de esta tarea. Muestre que, en este caso, el protocolo puede ser quebrado. En particular, muestre que esta tercera parte puede generar eficientemente un elemento $T \in G$ tal que $|\langle T \rangle| = q$, $T \neq g$ y, si T se utiliza en la generación de firmas de anillo, entonces puede determinar eficientemente quién generó cada firma.

Corrección. La asignación de puntajes para cada pregunta de este problema es la siguiente.

- (a) Tiene 0.3 puntos si la idea de su demostración es correcta pero tiene algunos errores menores, y tiene 0.5 puntos si su demostración está completa y correcta.
- (b) Tiene 0.3 puntos si da una idea de por qué el participante 1 no puede generar una firma válida, pero su idea está incompleta o tiene errores menores, y tiene 0.5 puntos si su idea está completa y correcta.
- (c) Tiene 0.3 puntos si la idea de su demostración es correcta pero tiene algunos errores menores, y tiene 0.5 puntos si su demostración está completa y correcta.

- (d) Tiene 0.3 puntos si explica cómo se puede determinar quién hizo cada firma, pero su idea está incompleta o tiene errores menores, y tiene 0.5 puntos si su explicación está completamente correcta.
2. [2 puntos] En esta pregunta usted deberá programar una clase que represente a un participante de un conjunto de personas que generan firmas de anillos trazables. También una clase que revise dichas firmas y que verifique si el participante que emitió la firma ya había emitido otra firma anteriormente. Utilizaremos siempre SHA256 y grupos multiplicativos de enteros módulo primos ($\mathbb{Z}_p^*$). En particular:
- Para transformar un número n a un string, simplemente utilizaremos `str(n)`.
 - Para transformar el hash de un string s a un elemento del grupo $\mathbb{Z}_p^*$ utilizaremos `int.from_bytes(sha256(s.encode())) % p`.

La clase que representa a un participante debe tener la siguiente interfaz:

```
class TraceableRingSignatureParticipant:
    def __init__(self, g: int, q: int, p: int) -> None:
        # Verify that p is prime, raise an AssertionError otherwise.
        # Verify that q is prime in  $\mathbb{Z}_p^*$ , raise an AssertionError
        # otherwise.
        # Verify that the order of the subgroup generated by g is q,
        # raise an
        # AssertionError
        # otherwise.
        # Randomly generate the participant's secret key x in {1..q}

    def get_public_key(self) -> int:
        # Returns the participant's public key ( $g^x \bmod p$ )

    def generate_traceable_ring_signature(self, public_keys: list[int],
                                         tag: str, message: str) ->
                                         Tuple[list[int], int, int]:
        # Compute  $T := (\text{SHA256}(p_1 || \dots || p_n || \text{tag})^r \bmod p)$ ,
        # where  $[p_1, \dots, p_n]$ 
        # is the list
        # public_keys of
        # public keys.
        # Verify that the order of the subgroup generated by T is q,
        # raise an
        # AssertionError
        # otherwise.
        # Generate a traceable ring signature for the given message,
        # tag and list of
        # public keys.
        # The output looks like  $([s_1, \dots, s_n], c_1, f)$ .
```

La clase que representa a un verificador debe tener la siguiente interfaz:

```
class Verifier:
    def __init__(self, g: int, q: int, p: int) -> None:
        # Verify that p is prime, raise an AssertionError otherwise.
        # Verify that q is prime in  $\mathbb{Z}_p^*$ , raise an AssertionError
        # otherwise.
```

```

        # Verify that the order of the subgroup generated by g is q,
        raise an
        AssertionError
        otherwise.

def verify_signature(self, public_keys: list[int], message: str, tag:
    str, signature: Tuple[list[
        int], int, int]) -> bool,
    bool:

    # Verify if the signature is a valid traceable ring signature
    for the given public
    keys, message and
    tag.

    # The first component of the output is True if the signature
    is valid.

    # The second component is True if this is the first signature
    of this participant
    seen by the verifier
    .

    # Note the verifier needs to store some state to compute the
    second component.

```

Corrección. La corrección de esta pregunta va a estar basada en el siguiente conjunto de tests.

- [0.25 puntos] Al inicializar un `TraceableRingSignatureParticipant` con un valor de p que no es primo, se debe obtener una excepción.
- [0.25 puntos] Al inicializar un `TraceableRingSignatureParticipant` con un valor de p primo y un valor de q que no es primo, se debe obtener una excepción.
- [0.25 puntos] Al inicializar un `TraceableRingSignatureParticipant` con valores primos de p y q , y con un elemento $g \in \mathbb{Z}_p^*$ tal que $|\langle g \rangle| \neq q$, se debe obtener una excepción.
- [0.25 puntos] Al inicializar un `Verifier` con valores primos de p y q , y con un elemento $g \in \mathbb{Z}_p^*$ tal que $|\langle g \rangle| \neq q$, se debe obtener una excepción.
- [0.5 puntos] Para un conjunto válido de ℓ participantes inicializados con los mismos parámetros p , q y g , si uno de ellos genera una firma para una lista de claves públicas `public_keys`, un tag `tag` y un mensaje `message`, entonces la firma debe tener la forma $([s_1, \dots, s_\ell], c_1, f)$ y el método `verify_signature` con estos parámetros debe retornar `(True, True)`.
- [0.25 puntos] Para un conjunto válido de ℓ participantes inicializados con los mismos parámetros p , q y g , si un mismo participante genera dos firmas válidas para mensajes distintos usando la misma lista de claves públicas `public_keys` y el mismo tag `tag`, entonces después de verificar la primera firma, el verificador debe retornar `(True, False)` al verificar la segunda firma.
- [0.25 puntos] Para una firma válida $\sigma = ([s_1, \dots, s_\ell], c_1, f)$ para una lista de claves públicas `public_keys`, un tag `tag` y un mensaje `message`, si se define una firma $\sigma' = ([s'_1, s_2, \dots, s_\ell], c_1, f)$ con $s'_1 \not\equiv_q s_1$, entonces `verify_signature(public_keys, message, tag,  $\sigma'$ )` debe rechazar la firma.